

Laboratory Report: Containerization with Docker

Experiment Title: Containerization with Docker

Data: 2026.6.3

Name: Lu Weichi

ID: 202383910009

Class:23Applied Computing

I. Experimental Purpose

Learn how to package a web application into a Docker Image.

Understand the concept of Immutable Infrastructure.

Practice mapping network ports between a Cloud Container and the Host.

II. Experiment Contents

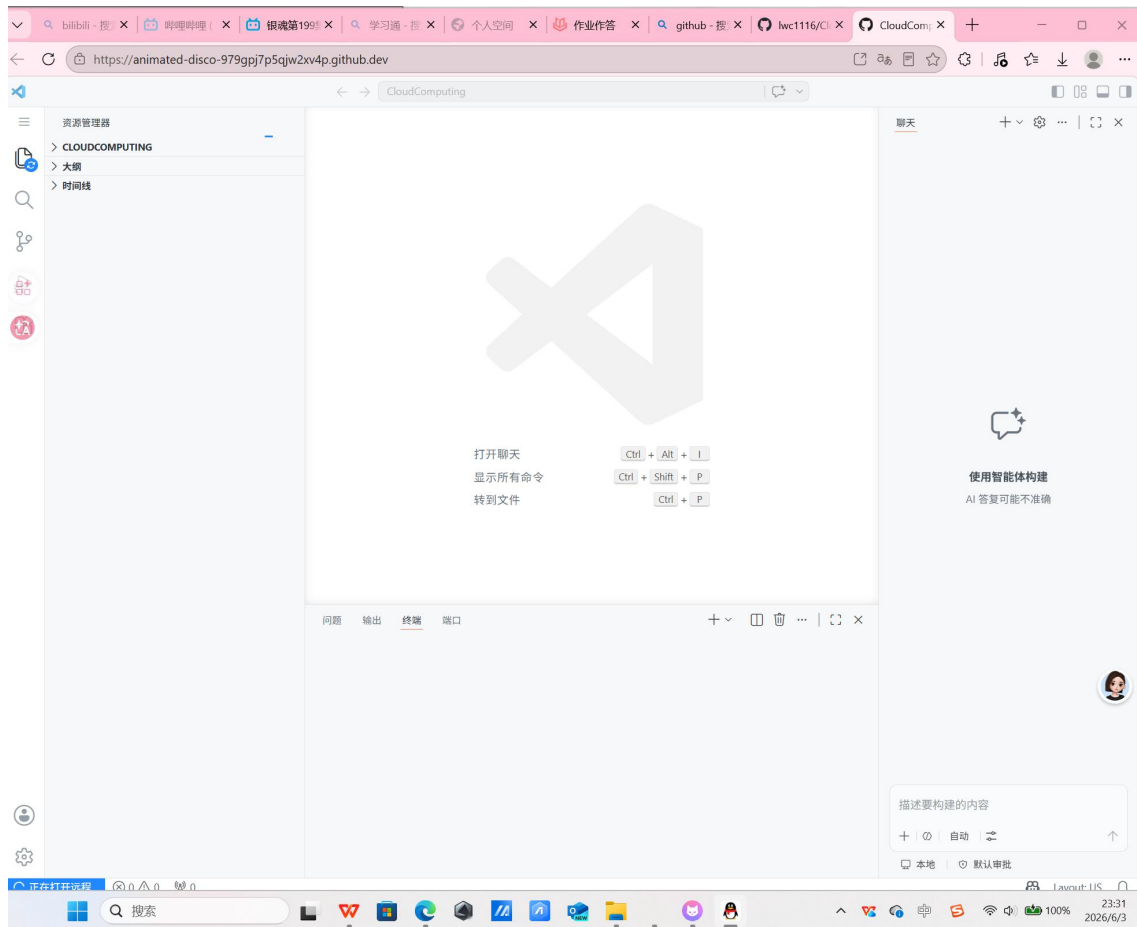
Build a Docker image for a static `index.html` file, run containers, test immutability, explore container internals, monitor resource usage, scale services, and clean up.

III. Detailed Steps and Observations

Step 0: Environment Setup – Launch GitHub Codespaces

Log into GitHub and open your repository.

Click the **Code** button → **Codespaces** → **Create codespace on main**.



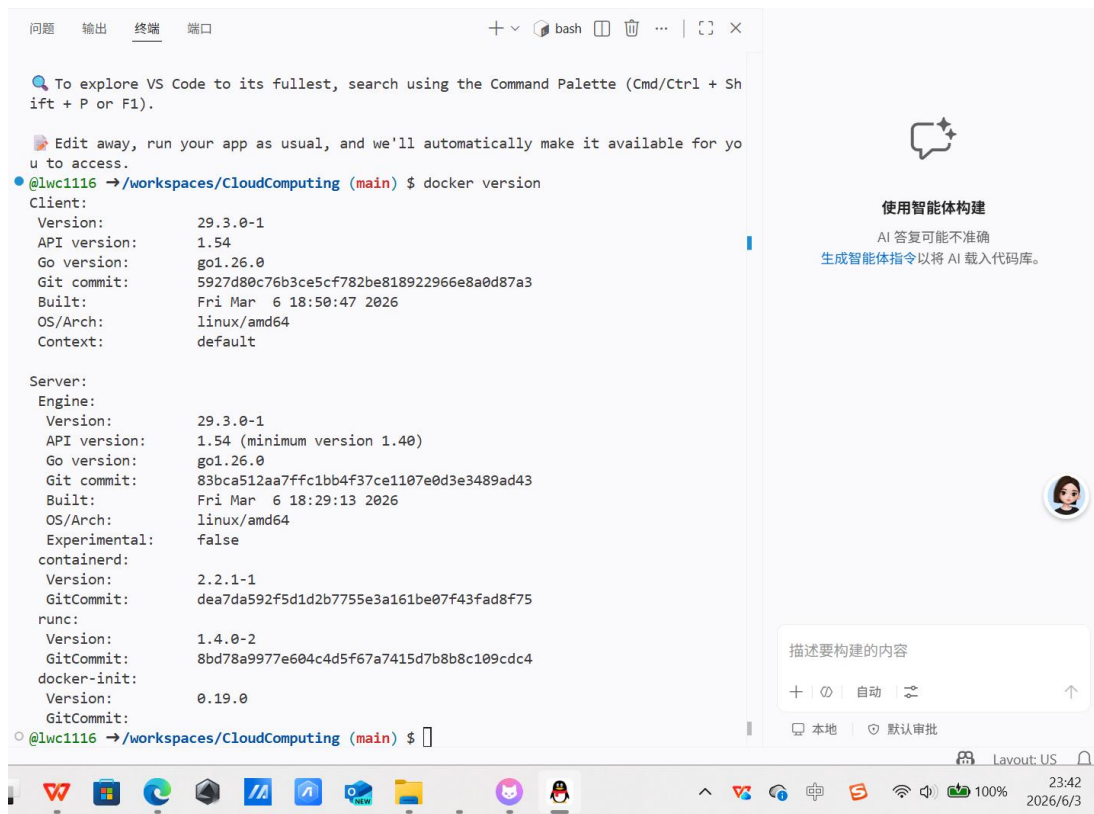
Wait for the Codespace to fully start (VS Code-like interface with a terminal at the bottom).

Step 1: Verify Docker Environment

In the Codespaces terminal, run:

`docker version`

Observation: Both Client and Server version information are displayed.



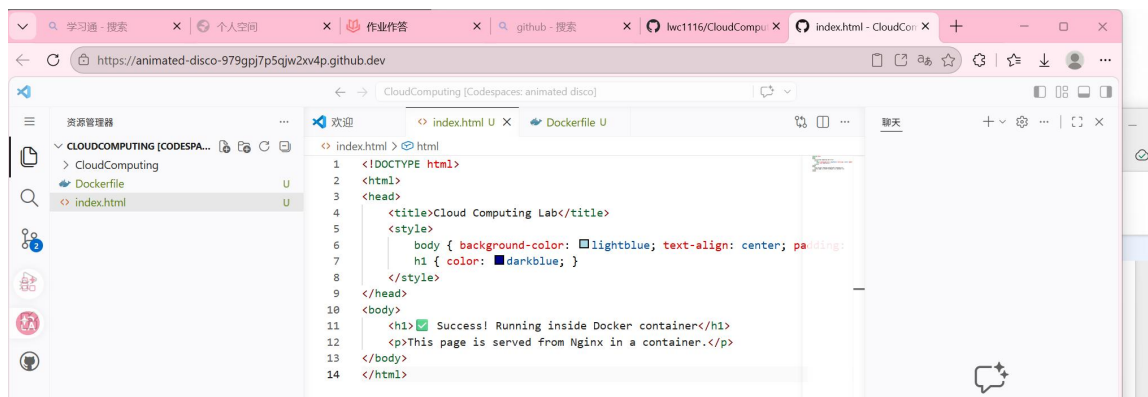
Step 2: Create a Dockerfile

In the left explorer, create a new file named exactly `Dockerfile` (no extension).

Paste the following content:

`dockerfile`

`FROM nginx:alpineCOPY index.html /usr/share/nginx/html/index.htmlEXPOSE 80`



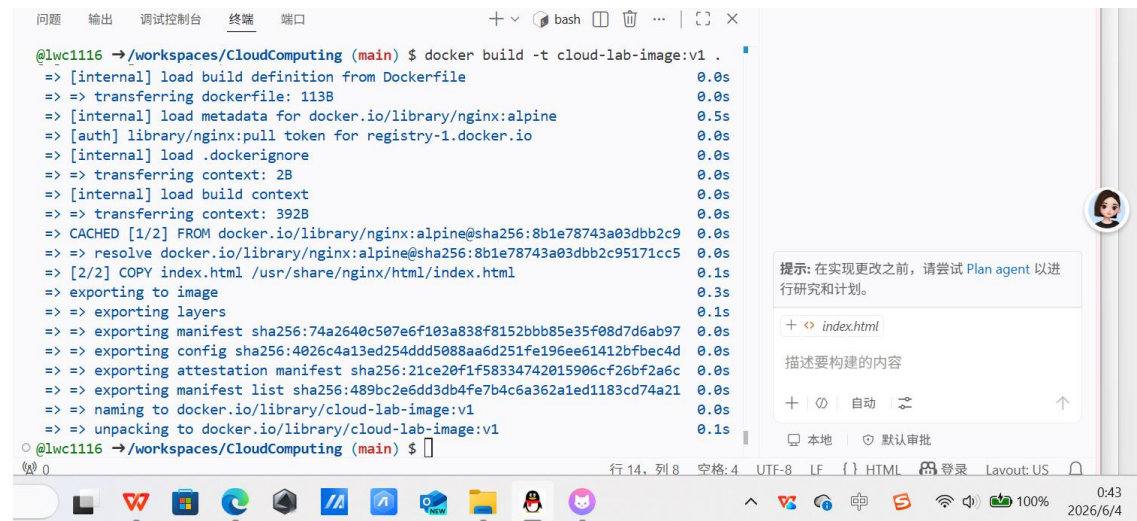
Step 3: Build the Docker Image

Run the build command (note the trailing dot):

```
bash
```

```
docker build -t cloud-lab-image:v1 .
```

Observation: Each step completes successfully and the final line shows `Successfully tagged cloud-lab-image:v1`.



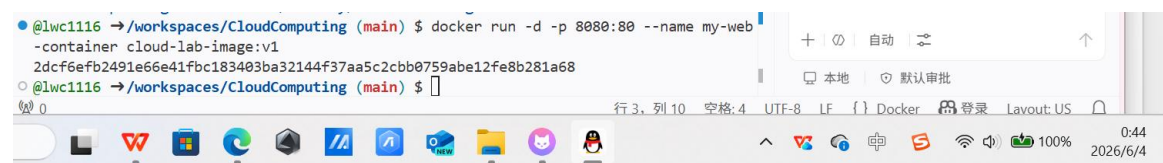
```
@lwc1116 →/workspaces/CloudComputing (main) $ docker build -t cloud-lab-image:v1 .
=> [internal] load build definition from Dockerfile 0.0s
=> => transferring dockerfile: 113B 0.0s
=> [internal] load metadata for docker.io/library/nginx:alpine 0.5s
=> [auth] library/nginx:pull token for registry-1.docker.io 0.0s
=> [internal] load .dockerignore 0.0s
=> => transferring context: 2B 0.0s
=> [internal] load build context 0.0s
=> => transferring context: 392B 0.0s
=> CACHED [1/2] FROM docker.io/library/nginx:alpine@sha256:8b1e78743a03dbb2c9 0.0s
=> => resolve docker.io/library/nginx:alpine@sha256:8b1e78743a03dbb2c95171cc5 0.0s
=> [2/2] COPY index.html /usr/share/nginx/html/index.html 0.1s
=> exporting to image 0.3s
=> => exporting layers 0.1s
=> => exporting manifest sha256:74a2640c507e6f103a838f8152bbb85e35f08d7d6ab97 0.0s
=> => exporting config sha256:4026c4a13ed254ddd5088aa6d251fe196ee61412bfbec4d 0.0s
=> => exporting attestation manifest sha256:21ce20f1f58334742015906cf26bf2a6c 0.0s
=> => exporting manifest list sha256:489bc2e6dd3db4fe7b4c6a362a1ed1183cd74a21 0.0s
=> => naming to docker.io/library/cloud-lab-image:v1 0.0s
=> => unpacking to docker.io/library/cloud-lab-image:v1 0.1s
@lwc1116 →/workspaces/CloudComputing (main) $
```

Step 4: Run the Container

Execute:

```
docker run -d -p 8080:80 --name my-web-container cloud-lab-image:v1
```

A long container ID is returned.

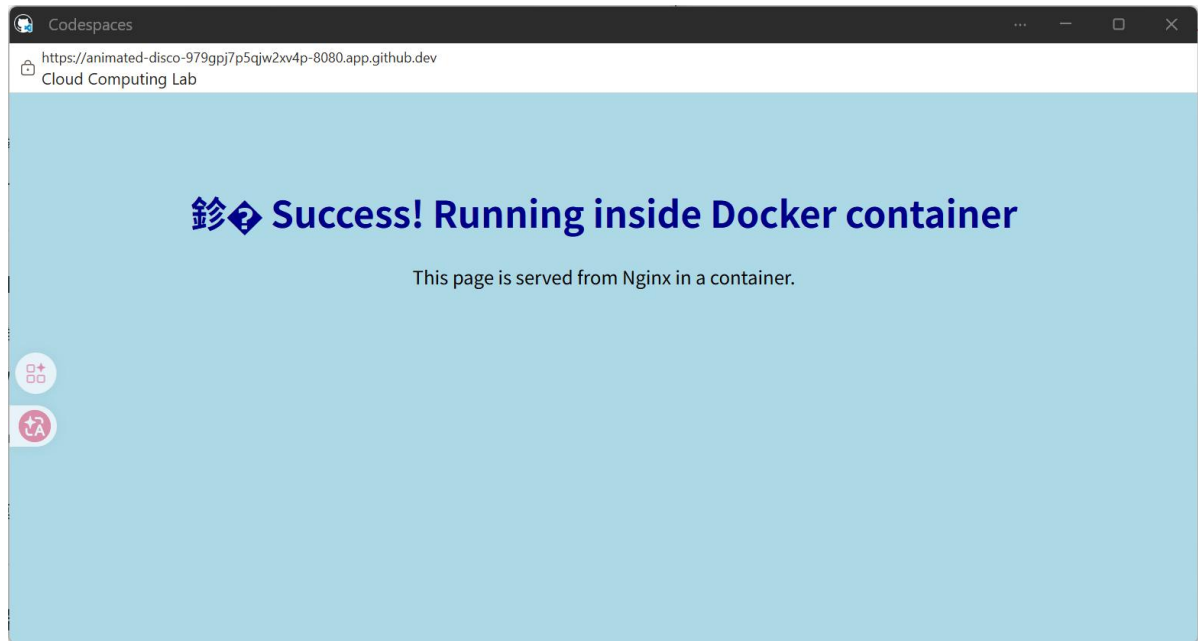


```
@lwc1116 →/workspaces/CloudComputing (main) $ docker run -d -p 8080:80 --name my-web
-container cloud-lab-image:v1
2dcf6efb2491e66e41fbc183403ba32144f37aa5c2cbb0759abe12fe8b281a68
@lwc1116 →/workspaces/CloudComputing (main) $
```

Step 5: Access the Web Server

Codespaces may show a pop-up: “A service is running on port 8080.” Click **Open in Browser**.

If not, manually open the **Ports** tab and click the globe icon next to port 8080.



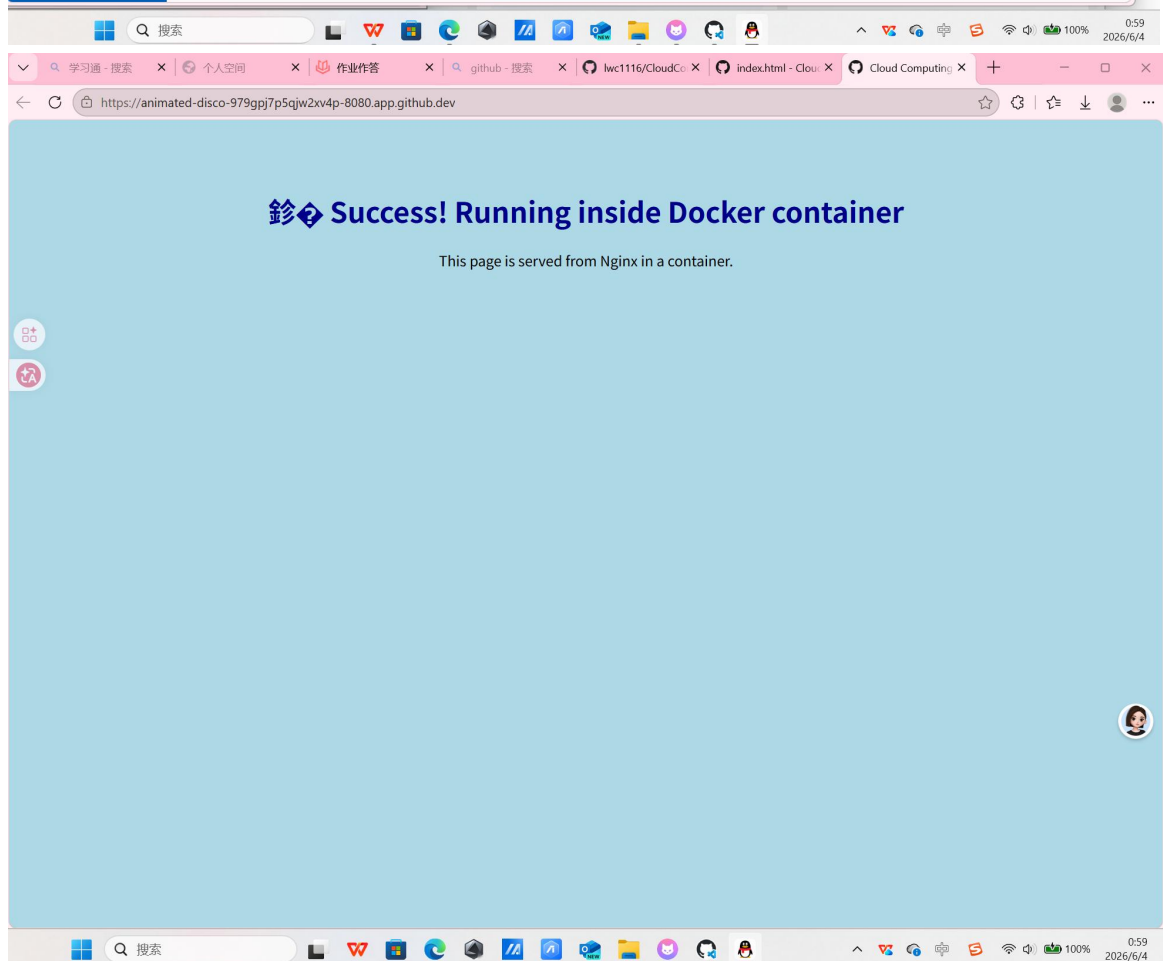
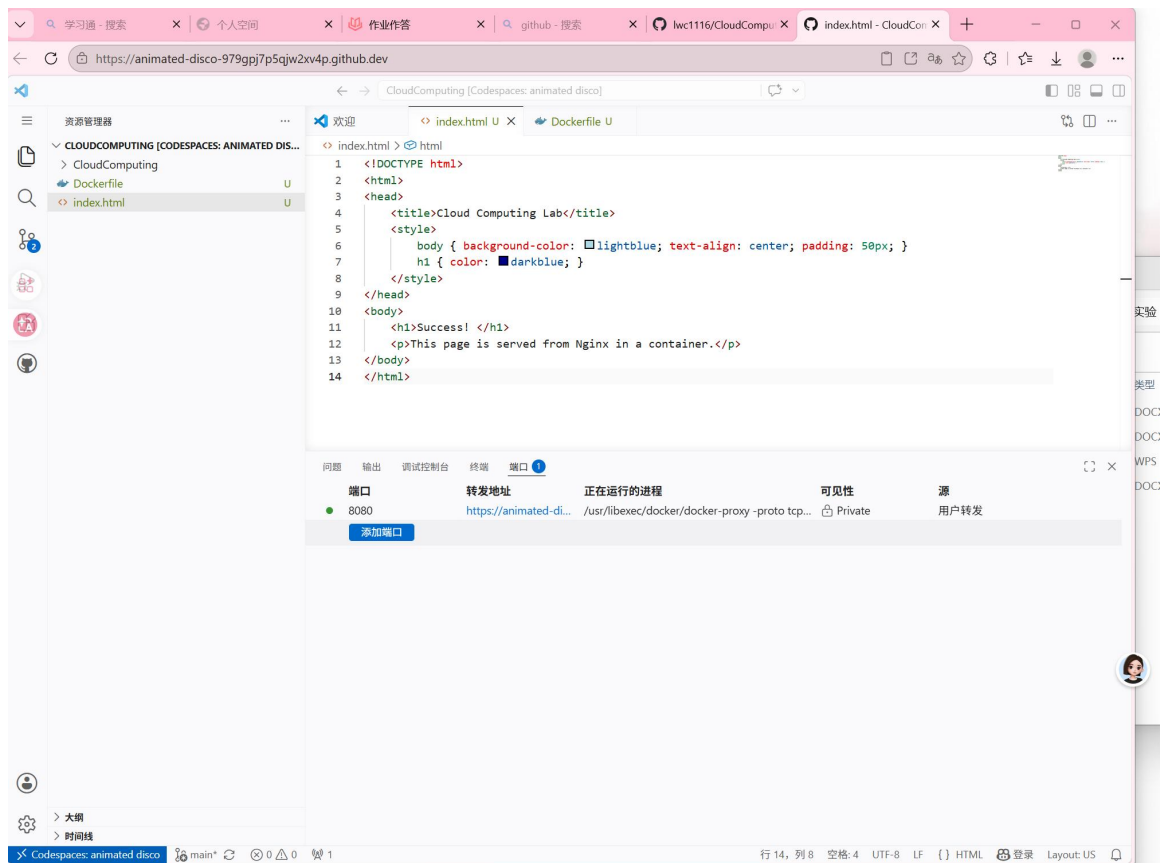
Step 6: Test Container Immutability

In the Codespaces explorer, open `index.html` and modify its content (e.g., change “Success” to “Modified”).

Save the file (Ctrl+S).

Refresh the browser tab (port 8080).

Observation: The page **does not change**; it still shows the old content.



Step 7: Explore the Container Interior

Enter the container:

```
docker exec -it my-web-container sh
```

Once the prompt changes to `#`, run:

```
ls /usr/share/nginx/html/cat /usr/share/nginx/html/index.html
```

Observation: The directory contains `index.html` and `cat` shows the **old** content (not the modified one).

```
@lwc1116 →/workspaces/CloudComputing (main) $ docker exec -it my-web-container sh
/ # ls /usr/share/nginx/html/
50x.html    index.html
/ # cat /usr/share/nginx/html/index.html
<!DOCTYPE html>
<html>
<head>
  <title>Cloud Computing Lab</title>
  <style>
    body { background-color: lightblue; text-align: center; padding: 50px; }
    h1 { color: darkblue; }
  </style>
</head>
<body>
  <h1>✅ Success! Running inside Docker container</h1>
  <p>This page is served from Nginx in a container.</p>
</body>
</html>/ #
```

Exit the container:

```
exit
```

Step 8: Resource Monitoring (Lightweight Nature)

Run:

```
docker stats
```

Observation: The Nginx container typically uses less than 10 MB of RAM.

CONTAINER ID	NAME	CPU %	MEM USAGE / LIMIT	MEM %	NET I/O	BLOCK I/O	PIDS
e58ba20bd1ac	my-web-container	0.00%	3.352MiB / 7.758GiB	0.04%	8.37kB / 5.03kB	8.19kB / 24.6kB	3

Press `Ctrl+C` to stop monitoring.

Step 9: Scaling and Port Management

Run a second container on host port 8081:

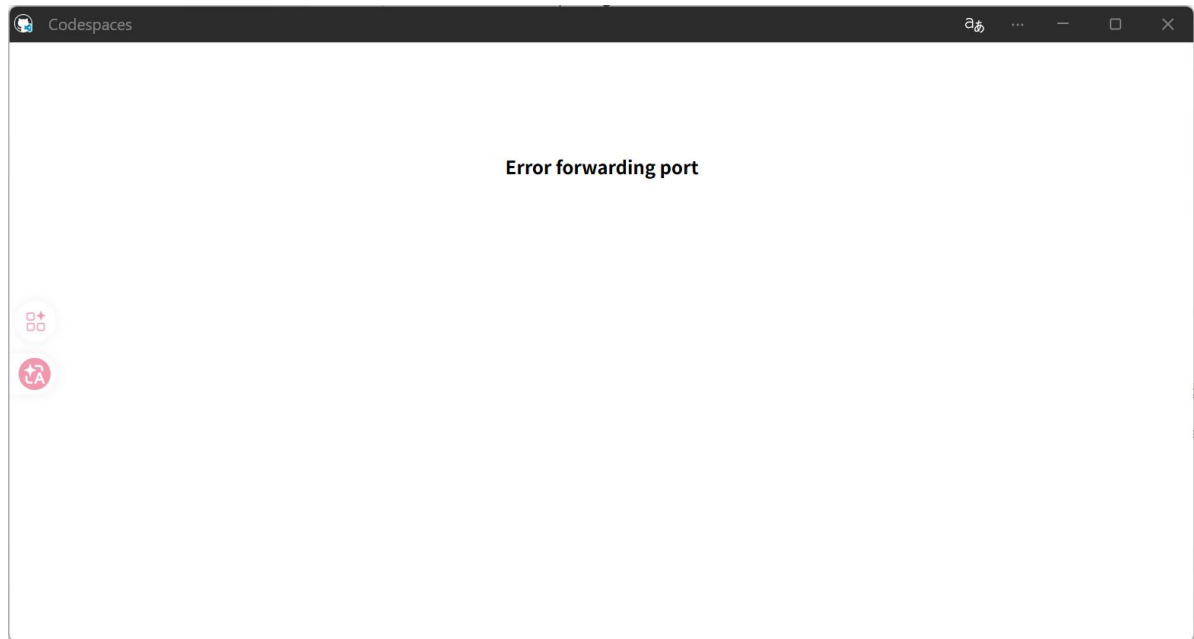
```
docker run -d -p 8081:80 --name second-container cloud-lab-image:v1
```

Open the **Ports** tab in Codespaces – now both 8080 and 8081 are listed.

```

● ^C@lwc1116 →/workspaces/CloudComputing (main) docker run -d -p 8081:80 --name second-container cloud-lab-image:v1v1
808973cbc913360602c99e239a36b9d4536dce77cc3501d97c9206da0efe579f
○ @lwc1116 →/workspaces/CloudComputing (main) $ 

```



Step 10: Lab Cleanup

Stop and remove both containers:

```
docker stop my-web-container second-container  
docker rm my-web-container second-container
```

Verify no containers remain:

```
docker ps
```

```

● @lwc1116 →/workspaces/CloudComputing (main) $ docker stop my-web-container second-container
my-web-container
second-container
● @lwc1116 →/workspaces/CloudComputing (main) $ docker rm my-web-container second-container
my-web-container
second-container
● @lwc1116 →/workspaces/CloudComputing (main) $ docker ps
CONTAINER ID   IMAGE     COMMAND                  CREATED        STATUS        PORTS   NAMES

```


Step 11: Save Changes to GitHub

git add .git commit -m "Finished Lab 02 - Docker containerization"git push

```
• @lwc1116 →/workspaces/CloudComputing (main) $ git add .
• @lwc1116 →/workspaces/CloudComputing (main) $ git commit -m "Finished Lab 02 - Docker containerization"
[main 30c4f2a] Finished Lab 02 - Docker containerization
 2 files changed, 17 insertions(+)
   create mode 100644 Dockerfile
   create mode 100644 index.html
• @lwc1116 →/workspaces/CloudComputing (main) $ git push
Enumerating objects: 5, done.
Counting objects: 100% (5/5), done.
Delta compression using up to 2 threads
Compressing objects: 100% (4/4), done.
Writing objects: 100% (4/4), 641 bytes | 641.00 KiB/s, done.
Total 4 (delta 0), reused 0 (delta 0), pack-reused 0 (from 0)
To https://github.com/lwc1116/CloudComputing
   e52ee30..30c4f2a  main -> main
○ @lwc1116 →/workspaces/CloudComputing (main) $ █
```

IV. Reflection

1. Problems Encountered and Proposed Solutions

Problem 1: During `docker build`, the error `"/index.html": not found` appeared.

Solution: The `index.html` file was missing in the workspace. I created a simple `index.html` file with basic HTML content and placed it in the same directory as the `Dockerfile`. After that, the build succeeded.

Problem 2: The Ports panel in Codespaces did not automatically show port 8080, and there was no “Add Port” button visible.

Solution: I manually constructed the preview URL using the codespace name (e.g., `https://animated-disco-xxxx-8080.preview.app.github.dev`) and accessed the webpage directly. Alternatively, using the `gh codespace ports forward` command can also forward the port.

Problem 3: Accidentally deleted the container before completing tests.

Solution: I re-ran `docker run -d -p 8080:80 --name my-web-container cloud-lab-image:v1` to recreate the container and continued the experiment.

2. Why did the website not update automatically after editing the code, and why is this actually a “feature” in cloud computing?

Answer:

The website did not update because a Docker container is **immutable** – the running container is a snapshot of the image at the time it was built. Changing the source file on the host does not affect the files inside the already running container; the `COPY` instruction in the `Dockerfile` only copies files during image build.

Why this is a feature in cloud computing:

Consistency: The same image produces identical behavior in development, testing, and production environments, eliminating the “works on my machine” problem.

Reliability: Running instances do not drift over time because no live changes can be made inside containers. Updates require a deliberate rebuild and redeployment.

Rollback ability: Old images can be instantly redeployed to revert to a previous state.

Scalability: Immutable images allow you to spin up hundreds of identical containers without worrying about configuration drift or unexpected modifications.

Thus, immutability is not a bug but a core design principle that enables predictable, secure, and scalable cloud operations.